CUDA Programming Model - Grid, Block, Thread, Warp

학습 목표

- Grid, Block, Thread 계층 구조 완벽 이해
- **Warp** 개념 (CUDA 성능의 핵심!)
- SM (Streaming Multiprocessor) 아키텍처
- Occupancy와 성능 관계
- 최적의 Kernel 구성 방법

예상 학습 시간: 4-5시간
난이도: ⭐ ⭐ (초급-중급)
선수 과목: 00_CUDA_Introduction.md

---

1. Thread Hierarchy 복습 및 심화

1.1 3단계 계층 구조

```
┌──────────────────────────────────────────────┐
│                GRID                          │  ← Kernel 전체
│   ┌──────────┐   ┌──────────┐               │
│   │ Block 0  │   │ Block 1  │ ...           │
│   │┌────┐┌───┐│   │┌────┐┌───┐│               │
│   │└T0  └T1 ┘│   │└T0  └T1 ┘│               │
│   │└T2  └T3 ┘│   │└T2  └T3 ┘│               │
│   └──────────┘   └──────────┘               │
└──────────────────────────────────────────────┘

Grid → Block → Thread (위에서 아래로)
```

1.2 차원 지원: 1D, 2D, 3D

(1) 1D 구성 - Vector 연산

```
// Vector addition: N = 1024
int N = 1024;
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;  // = 4

vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

// Kernel 내부
__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    //        └─ Block 0: 0*256 + [0-255] = 0-255
    //        └─ Block 1: 1*256 + [0-255] = 256-511
    //        └─ Block 2: 2*256 + [0-255] = 512-767
    //        └─ Block 3: 3*256 + [0-255] = 768-1023

    if (tid < N) {
        C[tid] = A[tid] + B[tid];
    }
}
```

(2) 2D 구성 - Matrix 연산

```
// Matrix: 1024 × 1024
int width = 1024, height = 1024;
dim3 threadsPerBlock(16, 16);  // 16×16 = 256 threads/block
dim3 blocksPerGrid((width + 15) / 16, (height + 15) / 16);  // 64×64 = 4096 bloc

matrixMul<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, width, height);

// Kernel 내부
__global__ void matrixMul(float* A, float* B, float* C, int width, int height) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;  // X → 열
    int row = blockIdx.y * blockDim.y + threadIdx.y;  // Y → 행

    if (col < width && row < height) {
        float sum = 0.0f;
        for (int k = 0; k < width; k++) {
            sum += A[row * width + k] * B[k * width + col];
        }
        C[row * width + col] = sum;
```

```
    }
}
```

2D 인덱싱 시각화:

```
Block (2, 1)에서 Thread (3, 2):

col = blockIdx.x * blockDim.x + threadIdx.x
    = 2 * 16 + 3 = 35

row = blockIdx.y * blockDim.y + threadIdx.y
    = 1 * 16 + 2 = 18

1D index = row * width + col = 18 * 1024 + 35 = 18467
```

(3) 3D 구성 - Volume 연산

```
// Volume: 256 × 256 × 128 (CT scan, 유체 시뮬레이션 등)
dim3 volume_size(256, 256, 128);
dim3 threadsPerBlock(8, 8, 8);  // 8×8×8 = 512 threads/block
dim3 blocksPerGrid((volume_size.x + 7) / 8,
                   (volume_size.y + 7) / 8,
                   (volume_size.z + 7) / 8);  // 32×32×16 blocks

volumeFilter<<<blocksPerGrid, threadsPerBlock>>>(d_vol, d_output, volume_size);

// Kernel 내부
__global__ void volumeFilter(float* vol, float* output, dim3 size) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

    if (x < size.x && y < size.y && z < size.z) {
        int idx = z * (size.x * size.y) + y * size.x + x;
        output[idx] = applyFilter(vol, x, y, z, size);
    }
}
```

1.3 Block 제약 조건

```
// Block당 최대 thread 개수
// - Compute Capability 2.x 이상: 1024 threads
```

```
// - Compute Capability 1.x: 512 threads

// ✅ 가능한 구성들
dim3 block1(1024, 1, 1);    // 1024 threads
dim3 block2(32, 32, 1);     // 32×32 = 1024 threads
dim3 block3(16, 16, 4);     // 16×16×4 = 1024 threads
dim3 block4(8, 8, 16);      // 8×8×16 = 1024 threads

// ❌ 불가능한 구성들
dim3 block_bad1(1025, 1, 1);     // 1025 > 1024
dim3 block_bad2(32, 33, 1);      // 32×33 = 1056 > 1024

// 각 차원의 최대값
// - x: 1024
// - y: 1024
// - z: 64

dim3 block_bad3(1024, 2, 1);  // x는 OK지만 total = 2048 > 1024
dim3 block_bad4(1, 1, 128);   // z = 128 > 64
```

---

2. Warp: CUDA의 핵심 비밀 🔑

2.1 Warp란 무엇인가?

Warp: GPU에서 실제로 **함께 실행되는 32개 thread의 그룹**

```
Block (256 threads) → 8개 Warp로 나뉨
┌──────────────────────────────────┐
| Warp 0: Thread  0 -  31          | ← 이 32개 threads가 "lock-step"
| Warp 1: Thread 32 -  63          |    (같은 명령어를 동시 실행)
| Warp 2: Thread 64 -  95          |
| Warp 3: Thread 96 - 127          |
| Warp 4: Thread 128- 159          |
| Warp 5: Thread 160- 191          |
| Warp 6: Thread 192- 223          |
| Warp 7: Thread 224- 255          |
└──────────────────────────────────┘
```

핵심 규칙:

- Warp 크기 = **항상 32** (모든 NVIDIA GPU)
- Warp ID = `threadIdx.x / 32` (1D 기준)
- Warp 내 lane ID = `threadIdx.x % 32`

2.2 SIMT (Single Instruction, Multiple Thread)

Warp의 모든 threads는 **같은 instruction**을 실행하지만, **다른 data**에 적용:

```
__global__ void example(int* data) {
    int tid = threadIdx.x;
    data[tid] = tid * 2;  // 모든 threads가 같은 명령어 실행
}

// Warp 0 실행 (threads 0-31):
// 시각화 - 모든 threads가 동시에 "tid * 2" 계산
Thread 0:  data[0]  = 0  * 2 = 0    ┐
Thread 1:  data[1]  = 1  * 2 = 2     │
Thread 2:  data[2]  = 2  * 2 = 4     ├─ 한 번에 실행!
...                                   │
Thread 31: data[31] = 31 * 2 = 62   ┘
```

2.3 Warp Divergence (성능 킬러!)

문제 상황: 조건문으로 인한 분기

```
__global__ void divergent_kernel(int* data) {
    int tid = threadIdx.x;

    if (tid % 2 == 0) {
        // Path A: 짝수 threads
        data[tid] = expensive_computation_A();
    } else {
        // Path B: 홀수 threads
        data[tid] = expensive_computation_B();
    }
}

// Warp 0 (threads 0-31) 실행 과정:
// Step 1: 짝수 threads만 실행 (홀수는 대기)
Threads  0,  2,  4, ..., 30 → Path A 실행
Threads  1,  3,  5, ..., 31 → ⏸ Idle (낭비!)
```

```
// Step 2: 홀수 threads만 실행 (짝수는 대기)
Threads  0,  2,  4, ..., 30 → ⏸ Idle (낭비!)
Threads  1,  3,  5, ..., 31 → Path B 실행

// 결과: 2배 시간 소요!
```

성능 영향 측정

```
// Divergent 버전 (나쁜 예)
__global__ void bad_divergence(float* out, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {
        if (tid % 2 == 0) {
            out[tid] = sqrt((float)tid);  // Path A
        } else {
            out[tid] = cbrt((float)tid);  // Path B - 다른 연산!
        }
    }
}

// Non-divergent 버전 (좋은 예)
__global__ void good_no_divergence(float* out, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {
        // 모든 threads가 같은 연산
        out[tid] = sqrt((float)tid);
    }
}

// 벤치마크 결과 (N = 1M):
// bad_divergence:      5.2 ms
// good_no_divergence:  2.8 ms
// → 1.86x slowdown due to divergence!
```

Divergence 최소화 전략

전략 1: Warp 단위로 작업 분할

```
// ❌ 나쁜 방법 - Warp 내부 divergence
__global__ void bad(int* data) {
    int tid = threadIdx.x;
    if (tid % 2 == 0) {
        // ...
    }
```

```
}

// ✅ 좋은 방법 - Warp 단위로 분할
__global__ void good(int* data) {
    int tid = threadIdx.x;
    int warp_id = tid / 32;

    if (warp_id % 2 == 0) {
        // Warp 0, 2, 4, ... → 전체 warp가 같은 경로
    } else {
        // Warp 1, 3, 5, ... → 전체 warp가 같은 경로
    }
}
```

전략 2: 조건문 제거 (Predication)

```
// ❌ 조건문 사용
if (condition) {
    result = a + b;
} else {
    result = a - b;
}

// ✅ 조건 연산자 (컴파일러가 predicated instruction 생성)
result = condition ? (a + b) : (a - b);
// 또는
result = a + (condition ? b : -b);
```

2.4 Warp-Level Primitives (고급)

(1) Warp Shuffle - 레지스터 간 직접 통신

```
// __shfl_down_sync: 아래쪽 lane에서 값 가져오기
__device__ float warp_reduce_sum(float val) {
    // Warp 내 모든 값을 합산 (shared memory 불필요!)
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
        // offset=16: lane 0은 lane 16의 값을 받음
        // offset=8:  lane 0은 lane 8의 값을 받음
        // ...
    }
    return val;  // lane 0만 최종 결과 보유
}
```

```
// 시각화 (초기값 모두 1.0):
// Step 1 (offset=16):
lane 0:  1 + lane[16] → 2
lane 1:  1 + lane[17] → 2
...
lane 15: 1 + lane[31] → 2

// Step 2 (offset=8):
lane 0:  2 + lane[8]  → 4
...

// Step 3 (offset=4):
lane 0:  4 + lane[4]  → 8

// Step 4 (offset=2):
lane 0:  8 + lane[2]  → 16

// Step 5 (offset=1):
lane 0: 16 + lane[1]  → 32 (완료!)
```

성능 비교:

```
Shared Memory 기반 reduction:  ~50 cycles
Warp Shuffle 기반 reduction:   ~10 cycles
Speedup: 5x! 🚀
```

(2) Warp Vote Functions

```
// 전체 warp가 조건을 만족하는지 체크
__global__ void check_convergence(float* data, float threshold) {
    int tid = threadIdx.x;
    float val = data[tid];

    // 모든 lane이 threshold 이하인가?
    bool all_converged = __all_sync(0xffffffff, val < threshold);

    if (all_converged) {
        // 조기 종료 가능 (warp 전체가 수렴)
    }
}
```

---

3. SM (Streaming Multiprocessor) 아키텍처

3.1 GPU 물리 구조

```
GPU (예: RTX 4090)
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐     │
│  │  SM 0   │  │  SM 1   │  │  SM 2   │  │  SM 3   │     │
│  └─────────┘  └─────────┘  └─────────┘  └─────────┘     │
│  ┌─────────┐  ┌─────────┐          ...                  │
│  │  SM 4   │  │  SM 5   │  (총 128개 SMs)               │
│  └─────────┘  └─────────┘                               │
│                                                          │
│  ┌──────────────────────────────────────────────────┐   │
│  │     Global Memory (24 GB GDDR6X)                 │   │
│  └──────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

3.2 SM 내부 구조 (Ampere/Ada 기준)

```
SM (Streaming Multiprocessor)
┌──────────────────────────────────────────────────┐
│  Warp Scheduler × 4                              │  ← 4개 warp 동시 발령
│  ┌──────────┐  ┌──────────┐  ┌──────────┐       │
│  │ Warp 0   │  │ Warp 1   │  │ Warp 2   │ ...   │
│  └──────────┘  └──────────┘  └──────────┘       │
│                                                  │
│  ┌──────────────────────────────────────────┐    │
│  │  CUDA Cores (FP32) × 128                 │    │  ← INT32/FP32 연산
│  ├──────────────────────────────────────────┤    │
│  │  Tensor Cores × 4                        │    │  ← FP16/BF16 matrix ops
│  ├──────────────────────────────────────────┤    │
│  │  Special Function Units (SFU) × 32       │    │  ← sin, cos, sqrt, etc.
│  ├──────────────────────────────────────────┤    │
│  │  Load/Store Units × 32                   │    │  ← 메모리 액세스
│  └──────────────────────────────────────────┘    │
│                                                  │
│  ┌──────────────────────────────────────────┐    │
│  │  Register File (65536 × 32-bit)          │    │  ← Thread 레지스터
│  ├──────────────────────────────────────────┤    │
│  │  Shared Memory / L1 Cache (128 KB)       │    │  ← 공유 메모리
```

```
│   └──────────────────────────────────────┘   │
│                                              │
│   Max Warps: 64 (= 2048 threads)             │
│   Max Blocks: 32                             │
└──────────────────────────────────────────────┘
```

3.3 Block → SM 할당

```
// 예시: 100개 blocks 실행
myKernel<<<100, 256>>>(args);

// RTX 4090: 128 SMs
// → 첫 번째 wave: 100 blocks가 100개 SM에 할당
// → 나머지 28개 SM은 대기 (리소스 낭비!)

// 더 나은 구성: SM 개수의 배수
int num_sms = 128;  // deviceQuery로 확인
int blocks = num_sms * 4;  // = 512 blocks
myKernel<<<blocks, 256>>>(args);
// → 각 SM이 4개 blocks씩 처리 (균등 분배)
```

SM 개수 확인 방법

```
int device;
cudaGetDevice(&device);

cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, device);

printf("SM count: %d\n", prop.multiProcessorCount);
printf("Max threads per SM: %d\n", prop.maxThreadsPerMultiProcessor);
printf("Max blocks per SM: %d\n", prop.maxBlocksPerMultiProcessor);
```

---

4. Occupancy: SM 활용도

4.1 Occupancy 정의

Occupancy = (Active Warps per SM) / (Max Warps per SM)

```
예시 1: 낮은 Occupancy (25%)
SM의 Max Warps = 64
Active Warps = 16
Occupancy = 16 / 64 = 25%
→ SM 리소스의 75%가 유휴 상태!

예시 2: 높은 Occupancy (100%)
Active Warps = 64
Occupancy = 64 / 64 = 100%
→ SM 완전 활용!
```

4.2 Occupancy를 제한하는 요인들

(1) Block당 Thread 개수

```
// SM당 Max Warps = 64 (2048 threads)

// 경우 1: threadsPerBlock = 128 (4 warps)
// → SM당 최대 16 blocks (16 × 4 = 64 warps) ✅ 100% occupancy

// 경우 2: threadsPerBlock = 1024 (32 warps)
// → SM당 최대 2 blocks (2 × 32 = 64 warps) ✅ 100% occupancy

// 경우 3: threadsPerBlock = 512 (16 warps)
// → SM당 최대 4 blocks (4 × 16 = 64 warps) ✅ 100% occupancy

// 경우 4: threadsPerBlock = 96 (3 warps)
// → SM당 최대 21 blocks (21 × 3 = 63 warps) ❌ 98% occupancy
//   (64는 3의 배수가 아니므로 1 warp 낭비)
```

(2) Register 사용량

```
// SM당 Register File = 65536 개 (32-bit registers)

__global__ void heavy_register_kernel() {
    float a, b, c, d, e, f, g, h;  // 8 registers
    float i, j, k, l, m, n, o, p;  // 8 registers
    // ... 복잡한 계산 ...
    // Total: ~50 registers per thread
}

// Thread당 50 registers 사용
```

```
// → SM당 최대 threads = 65536 / 50 = 1310 threads
// → Max warps = 1310 / 32 = 40 warps
// → Occupancy = 40 / 64 = 62.5% (레지스터 부족으로 제한!)
```

해결 방법:

```
# 컴파일 시 레지스터 사용량 제한
nvcc -maxrregcount=32 kernel.cu

# 레지스터 사용량 확인
nvcc --ptxas-options=-v kernel.cu
# 출력: ptxas info : Used 48 registers, 1024 bytes smem
```

(3) Shared Memory 사용량

```
// SM당 Shared Memory = 128 KB (Ampere/Ada, configurable)

__global__ void heavy_smem_kernel() {
    __shared__ float tile[64][64];  // 64×64×4 = 16 KB
    // ...
}

// Block당 16 KB shared memory 사용
// → SM당 최대 blocks = 128 KB / 16 KB = 8 blocks
// → threadsPerBlock = 256이면:
//   Active warps = 8 blocks × 8 warps/block = 64 warps ✅ 100%
// → threadsPerBlock = 128이면:
//   Active warps = 8 blocks × 4 warps/block = 32 warps ❌ 50%
```

4.3 Occupancy Calculator 사용

방법 1: CUDA Toolkit 포함 Excel

```
# Linux
/usr/local/cuda/tools/CUDA_Occupancy_Calculator.xls

# 입력:
# - Compute Capability: 8.9 (RTX 4090)
# - Threads per block: 256
# - Registers per thread: 32
# - Shared memory per block: 8192 bytes
```

```
# 출력:
# - Theoretical Occupancy: 100%
# - Active Warps: 64
# - Active Threads: 2048
```

방법 2: CUDA API (Runtime)

```
int blockSize = 256;
int minGridSize, bestBlockSize;

cudaOccupancyMaxPotentialBlockSize(&minGridSize, &bestBlockSize,
                                   myKernel, 0, 0);

printf("Suggested block size: %d\n", bestBlockSize);

// 실제 occupancy 계산
int numBlocks;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, myKernel,
                                              blockSize, 0);
printf("Active blocks per SM: %d\n", numBlocks);

float occupancy = (numBlocks * blockSize / 32) / 64.0f;
printf("Occupancy: %.1f%%\n", occupancy * 100);
```

4.4 높은 Occupancy = 빠른 성능? (주의!)

오해: Occupancy 100%면 최고 성능!
진실: 항상 그렇지는 않습니다.

```
// 예시: Memory-bound kernel
__global__ void memory_heavy(float* data, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {
        data[tid] = data[tid] * 2.0f;  // 단순 연산, 메모리 대역폭 한계
    }
}

// Occupancy 50% (32 warps):  150 GB/s
// Occupancy 100% (64 warps): 155 GB/s (큰 차이 없음!)
// → 메모리 대역폭이 이미 포화 상태
```

교훈:

- Compute-bound: Occupancy ↑ → 성능 ↑
- Memory-bound: Occupancy 영향 적음
- 실험적으로 최적값 찾기!

---

5. 최적의 Kernel 구성 전략

5.1 Block Size 선택 가이드

Rule of Thumb:

```
// 권장: 128 - 512 threads per block

// ✅ 좋은 선택들
int blockSize = 128;  // 4 warps
int blockSize = 256;  // 8 warps (가장 흔함)
int blockSize = 512;  // 16 warps

// ❌ 피해야 할 선택들
int blockSize = 64;   // 2 warps - 너무 적음 (overhead 큼)
int blockSize = 1024; // 32 warps - 레지스터/smem 압박
int blockSize = 100;  // warp(32)의 배수가 아님!
```

이유:

```
blockSize = 64 (2 warps):
- Block 생성/관리 overhead가 상대적으로 큼
- Warp scheduler 효율 낮음

blockSize = 256 (8 warps): ✅ Sweet Spot
- Block overhead 적절
- 충분한 warp로 latency hiding
- 레지스터/smem 압박 적음

blockSize = 1024 (32 warps):
- SM당 최대 2 blocks만 가능 (max blocks = 32 제약)
- 레지스터/smem 경쟁 심함
- Flexibility 낮음
```

5.2 Grid Size 선택

```
int N = 1000000;  // 데이터 크기
int blockSize = 256;

// 방법 1: 정확히 필요한 만큼
int gridSize = (N + blockSize - 1) / blockSize;  // = 3907 blocks

// 방법 2: SM의 배수로 (load balancing)
int num_sms = 128;  // RTX 4090
int gridSize = num_sms * 4;  // = 512 blocks
// 각 thread가 여러 원소 처리 (grid-stride loop)

// 방법 3: API로 자동 계산
int minGridSize, optimalBlockSize;
cudaOccupancyMaxPotentialBlockSize(&minGridSize, &optimalBlockSize,
                                   myKernel, 0, 0);
```

5.3 Grid-Stride Loop 패턴

```
// 전통적 방법 (문제: grid size가 데이터 크기에 종속)
__global__ void traditional(float* data, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {
        data[tid] *= 2.0f;
    }
}

// Grid-stride loop (권장: 유연하고 효율적)
__global__ void grid_stride(float* data, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;  // 총 thread 개수

    for (int i = tid; i < N; i += stride) {
        data[i] *= 2.0f;
    }
}

// 사용 (grid size 자유롭게 선택 가능)
int blockSize = 256;
int gridSize = 32 * num_sms;  // 적당한 크기
grid_stride<<<gridSize, blockSize>>>(d_data, N);
```

```
// 장점:
// 1. N이 얼마든 상관없음 (loop가 처리)
// 2. SM 활용도 최적화 가능
// 3. 각 thread가 cache-friendly 방식으로 여러 데이터 처리
```

---

6. 실습: Matrix Transpose

6.1 Naive 버전

```
// Naive matrix transpose (coalescing 미고려)
__global__ void transpose_naive(float* input, float* output,
                                int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height) {
        // Read:  input[y * width + x]  - coalesced (연속 메모리)
        // Write: output[x * height + y] - strided (비연속!)
        output[x * height + y] = input[y * width + x];
    }
}

// 실행
dim3 block(16, 16);  // 256 threads
dim3 grid((width + 15) / 16, (height + 15) / 16);
transpose_naive<<<grid, block>>>(d_in, d_out, width, height);

// 성능: ~80 GB/s (이론적 최대 대비 30%)
```

6.2 Shared Memory 최적화 버전

```
#define TILE_DIM 32

__global__ void transpose_shared(float* input, float* output,
                                 int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;
```

```
    // Coalesced read from global memory
    if (x < width && y < height) {
        tile[threadIdx.y][threadIdx.x] = input[y * width + x];
    }

    __syncthreads();  // 모든 threads가 shared memory 로드 완료 대기

    // Transpose coordinates
    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    // Coalesced write to global memory
    if (x < height && y < width) {
        output[y * height + x] = tile[threadIdx.x][threadIdx.y];
    }
}

// 실행
dim3 block(TILE_DIM, TILE_DIM);  // 1024 threads
dim3 grid((width + TILE_DIM - 1) / TILE_DIM,
          (height + TILE_DIM - 1) / TILE_DIM);
transpose_shared<<<grid, block>>>(d_in, d_out, width, height);

// 성능: ~200 GB/s (2.5배 향상!)
```

왜 빠른가?

1. Global memory: 항상 coalesced access (읽기/쓰기 모두)
2. Transpose 연산은 shared memory에서 수행 (빠름!)
3. Bank conflict만 없으면 최적!

---

7. 디버깅 팁

7.1 Thread 정보 출력

```
__global__ void debug_kernel() {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    if (tid == 0) {  // 첫 번째 thread만 출력
```

```
        printf("gridDim: (%d, %d, %d)\n", gridDim.x, gridDim.y, gridDim.z);
        printf("blockDim: (%d, %d, %d)\n", blockDim.x, blockDim.y, blockDim.z);
    }

    if (blockIdx.x == 0 && threadIdx.x < 4) {
        printf("Block 0, Thread %d: tid = %d\n", threadIdx.x, tid);
    }
}
```

7.2 Occupancy 프로파일링

```
# Nsight Compute로 occupancy 측정
ncu --metrics sm__warps_active.avg.pct_of_peak ./program

# 출력:
# sm__warps_active.avg.pct_of_peak: 85.2%
# → Occupancy가 85%임을 의미
```

---

8. 다음 단계

이제 Memory Hierarchy를 학습하세요:

- [02_Memory_Hierarchy.md]
  - Global, Shared, Register 메모리 성능 비교
  - Memory Coalescing (필수!)
  - Cache 활용 전략

실습 예제:

- `examples/cuda/01_matrix_multiply.cu` - 2D 인덱싱 마스터
- `examples/cuda/02_reduction.cu` - Warp primitives 활용

요약 체크리스트

- [ ] Grid, Block, Thread 계층 구조 완벽 이해

- [ ] **Warp (32 threads)** 개념 숙지
- [ ] Warp Divergence 문제와 해결법 이해
- [ ] SM 아키텍처와 block 할당 메커니즘 파악
- [ ] Occupancy 계산 가능
- [ ] 최적의 block size 선택 가능 (128-512)
- [ ] Grid-stride loop 패턴 작성 가능
- [ ] Matrix transpose 예제 이해 및 수정 가능

핵심 개념:

- Warp = 실제 실행 단위 (32 threads)
- Divergence = 성능 저하의 주범
- Occupancy ≠ 성능 (하지만 중요!)
- Block size = 256이 sweet spot (대부분의 경우)